

OpenGuide

开源项目新手贡献智能向导

输入一个 GitHub 仓库 URL，输出一份带证据、可执行、会陪你走完每一步的新手贡献路线图。

申报方向	方向一（开源赋能的 AI 应用创新）为主；方向三（真实开源贡献闭环）作佐证
作品名称	OpenGuide —— 开源项目新手贡献智能向导
团队	3 人（全栈，代号 A / B / C）
许可	MIT
文档版本	2026-10-06

目录

1. 作品简介与选题论证

- 1.1 问题
- 1.2 我们的答案
- 1.3 为什么这是国家级奖项的选题

2. 系统设计：四段管线

- 2.1 为什么不是「clone 全文 → LLM 写总结」
- 2.2 M1：四视角分层索引（技术差异点核心）
- 2.3 M2：证据锚点校验（防幻觉核心）
- 2.4 无 Key 也能跑（演示与离线评审友好）

3. 关键工程决策

4. 开源清单与引用

5. 验证证据：四层评测

- 5.1 五个指标（benchmark/metrics.py，规则化、确定性）
- 5.2 实测结果（2026-10-06，无 LLM Key 的确定性路径）
- 5.3 golden 仓库集
- 5.4 自动测试

6. 真实贡献记录（方向三佐证）

7. 运行与复现

- 7.1 一键部署（评审冷启动推荐）
- 7.2 本地开发
- 7.3 接口

8. 项目路线图与开源计划

- 8.1 已交付
- 8.2 后续计划
- 8.3 开源计划

9. 隐私与授权

附：文档索引

作品名称：OpenGuide —— 开源项目新手贡献智能向导 **申报方向：**方向一（开源赋能的 AI 应用创新）为主；方向三（真实开源贡献闭环）作佐证 **一句话定位：**输入一个 GitHub 仓库 URL，输出一份**带证据、可执行、会陪你走完每一步**的新手贡献路线图。

1. 作品简介与选题论证

1.1 问题

开源生态最大的摩擦不在"写代码"，而在**第一次贡献之前的那段路**：环境怎么搭、测试怎么跑、第一个 issue 怎么选、PR 怎么提。这些信息散落在 README、CONTRIBUTING、docs/、Makefile、CI workflow 与 issue 标签里，新手既不知道去哪找，也不知道找得对不对。

通用聊天机器人能给一段"看起来像那么回事"的答案，但它**无法保证答案来自这个仓库**—— 编造的安装命令、不存在的 issue 编号，正是新手最容易被坑的地方。

1.2 我们的答案

OpenGuide 把非结构化仓库加工成**结构化、可检索、带证据、可分步执行的操作手册**，并守住一条硬边界：

无证据不宣称。 每条建议都要能回溯到仓库里的真实文件或 issue；不能确认的，宁可标注 **missing** 并建议提问，也不编造安装命令或 issue 编号。

1.3 为什么这是国家级奖项的选题

维度	论证
痛点真实性	新人 onboarding 是开源生态公认瓶颈；评委多为开源从业者，共鸣天然
时代共振	"AI + 开源"是本届核心命题，本项目直接把"开源协作的入口问题"作为 AI 应用对象
差异化	不是"套 API 的通用聊天机器人"，产物是 可解释、可验证的操作手册 ，管线可见
可验证闭环	黄金评估集 + 证据锚点校验 + 真实 PR 贡献，三重验证（见 §5、§6）

2. 系统设计：四段管线

GitHub 仓库 URL



M0 仓库勘探与画像	backend/src/backend/core/github
只抓"决策相关"信号, 不 clone 全文	
→ 语言占比 / 构建面 / good-first-issue / 活跃度 / 协议	
→ 磁盘缓存 (规避限流)	

Profile (固定 schema)



M1 分层视角化索引	backend/src/backend/core/index
按"新手阅读路径"把仓库重组为四个视角, 各自独立建索引	
V1 入口与约定 / V2 架构地图 / V3 可执行路径 / V4 issue	

四视角索引



M2 结构化指南生成	backend/src/backend/core/generate
画像驱动决策树 → Stage A-D 分阶段检索 → schema 约束生成	
→ 证据锚点逐条校验 (不通过就降级为 missing)	

Guide (每步带 evidence)



M3 分步引导交互	frontend/src/components/GuideWizard
向导卡片流 + 三步追问 + 常驻证据面板	

2.1 为什么不是「clone 全文 → LLM 写总结」

朴素方案必败的三个失配，正是四段管线的设计依据：

失配	说明	对策
规模失配	数千文件塞不进上下文，噪声淹没信号	M0 只抓决策相关信号；M1 分视角建索引
信息类型失配	"如何搭环境"分散在 README / CONTRIBUTING / docs / workflow	V3 把可执行路径跨文件聚合到一起
能力前提失配	新手卡住时不会定位问题	指南带预期结果与失败分支；三步追问兜底

2.2 M1：四视角分层索引（技术差异点核心）

代码 chunk 对"如何上手"几乎无用；全库检索会把"搭环境"和"某行代码逻辑"混为一谈。我们按新手阅读路径重组为四个视角，各视角独立建索引：

视角	内容来源	回答什么
V1 入口与约定	README + CONTRIBUTING + 规范 + license	贡献流程、协议边界
V2 架构地图	目录树 + 顶层结构	代码怎么组织、改哪在哪
V3 可执行路径	构建清单 + Makefile/脚本 + 安装章节	怎么跑起来 / 跑测试（新手 90% 卡点）
V4 issue 槽位	good-first-issue 标题/正文/标签	现在能做什么

V3 的准入规则值得单说：一个文档只有**真的含命令行**（或本身就是构建文件）才进 V3。这条规则是被真实缺陷逼出来的——`psf/requests` 的 `docs/dev/contributing.rst` 是纯叙述文档（一条命令都没有），早期版本仅凭文件名含 `contribut` 就把它收进 V3，它靠 11 个分块把真正含 `python -m pytest tests` 的 `Makefile` 挤出了检索结果，导致“怎么跑测试”答不出来。同类被排除的还有 `changelog / history / authors`——它们是“变更与署名”，不是可执行路径。

2.3 M2：证据锚点校验（防幻觉核心）

生成不是“让模型自由发挥”，而是**先检索、再填空、最后审计：**

- 1. 每个 Stage 只拿到该阶段检索到的仓库原文片段；
- 2. 模型按严格 JSON schema 填写 `GuideStep` (`title / command / expected / fail_hints / evidence`) ；
- 3. **证据审计：**`evidence.source` 必须能解析到我们真实抓取过的文件路径或 issue 编号；

解析不了就降级为 `kind="missing"`——宁可承认“没找到出处”，也不留一条编造的断言；

- 1. 仍缺出处的步骤，再按阶段相关视角做一次定向回填。

2.4 无 Key 也能跑（演示与离线评审友好）

未配置大模型时，生成走**确定性启发式**路径：用规则从检索到的原文里挑出**属于该阶段**的真实命令（A 安装 / B 测试 / C issue / D git-PR），挑不到就留空。这条路让 CI 与离线评审无需任何密钥即可跑通全链路。

3. 关键工程决策

决策	理由
不 clone 全文，只走 GitHub API	省带宽、可缓存、能在无 git 环境运行；配合 TTL 磁盘缓存规避限流
向量检索可降级	chromadb 不可用或嵌入模型缺失时，回退到确定性的词元重叠检索，测试与演示在离线环境仍可跑
证据来源保留仓库真实大小写	前端据此拼“打开原始出处”链接，大小写错了会 404
意图路由用整词匹配	早期用子串匹配，短词 <code>pr</code> 会命中 <code>improve/progress</code> ，把问题错误路由到“贡献流程”
LLM 配置只存会话内存	Key 不落盘、不进日志；页面右上角可临时填写

4. 开源清单与引用

本项目以 MIT 许可发布（见 LICENSE）。不自称纯开源：编排、分层索引、证据校验、评测体系为团队自主实现；通用大模型走商业 API。

完整的第三方组件、开源模型与商业服务清单及其许可条件，逐项列于 third_party_disclosure.md。

5. 验证证据：四层评测

评测分四层，口径红线：仓库内样例/烟测不得宣称生成质量；任何质量数字必须能回溯到人工复核过的 golden 仓库集。详见 ../benchmark/README.md。

层	命令	作用	是否产出质量数字
L1 烟测	python -m benchmark.l1_smoke	离线验证 URL→画像→决策树→索引→生成→schema→证据 全链路通	否（明确不产出）
L2 黄金评估	python -m benchmark.l2_golden.runner	对 golden 仓库跑真实生成，按五维度打分	是（人工复核集）
L3 消融	python -m benchmark.l3_ablation.runner	同批仓库对比「朴素单次生成」vs「分层四视角管线」，含耗时	是（同基准防漂移）
L4 真实贡献	python -m benchmark.l4_contribution.runner	用本产品实际提交 PR 的记录	记录，非评分

5.1 五个指标（benchmark/metrics.py，规则化、确定性）

指标	含义
step_coverage 步骤完整率	生成的步骤覆盖了多少 golden 步骤
evidence_hit 证据命中率	生成步骤的证据来源能否解析到真实仓库文件/issue
command_correct 命令正确率	生成的命令是否与 golden 参考命令语义一致（"跑对没有"）
command_exec 命令可执行率	命令能否在真实 clone 上执行（"跑得动吗"）
assert_rate 有据断言率	非 missing 证据占全部步骤的比例——「无证据不宣称」的直接度量

另记录 单仓库生成耗时（elapsed_s），用于回答"分层的管线多换来了什么、多花了多少时间"。

5.2 实测结果（2026-10-06，无 LLM Key 的确定性路径）

完整归因与复现方式见 evaluation_report.md。

L2 指标	结果
步骤完整率	53%
证据命中率	75%
命令正确率	67%
命令可执行率	23%
有据断言率	85%

L3 消融（同批仓库 / 同 golden / 同指标）	naive 单次生成	分层四视角管线
证据命中率	10%	85%
步骤完整率	31%	53%
命令正确率	50%	67%

怎么读：朴素方案的"有据断言率"是 100%，但其证据命中率只有 10%——它声称的出处绝大多数是编的。分层管线经证据校验后证据命中率达到 85%，同时诚实性体现为"有据断言率"反而更低（验证不过的标 missing 而非硬编）。两个指标必须一起读。

低分项（如命令可执行率 23%）的逐条归因见评测报告 §3：聚合测试套件过重、命令依赖未声明的插件、测试本身依赖外部网络——**评测体系真的在起作用。**

5.3 golden 仓库集

5 个仓库均已人工复核并标 `verified: true`（见 `benchmark/golden/repos.json`）：

仓库	类型	覆盖特征
<code>psf/requests</code>	Python 库	文档最全、最典型的新手入口
<code>pallets/click</code>	Python CLI 框架	pyproject + pytest，构建面干净
<code>cookiecutter/cookiecutter</code>	项目脚手架	脚手架场景代表
<code>mochajs/mocha</code>	JavaScript 测试框架	跨语言 （package.json + npm test）
<code>camelot-dev/camelot</code>	PDF 表格抽取	依赖较重，验证鲁棒性

5.4 自动测试

套件	命令	数量
后端单测/集成	<code>python -m pytest backend/tests -q</code>	68
评测体系单测	<code>python -m pytest benchmark/tests -q</code>	14
合计	—	82
CI	<code>.github/workflows/ci.yml</code>	后端测试 + 前端构建 + 密钥扫描

CI 全程走 hermetic 路径（`USE_CHROMA=false`），**不下载任何嵌入模型**，秒级完成。

6. 真实贡献记录（方向三佐证）

L4 记录"用本产品自己走通一个真实 PR"的闭环证据，落盘于 `benchmark/l4_contribution/records.json`，用一行代码补录：

```
from benchmark.l4_contribution.runner import add_record
add_record("owner", "repo", "pr", "https://github.com/owner/repo/pull/123", "用 OpenGuide 走通的文档改进")
```

状态语义（避免"待补充"被误读成"失败"）：

- **PENDING**：尚无记录（退出码 0，明确标注待补充）
- **OK**：已有记录，逐条打印
- **FAIL**：记录文件损坏（真错误）

口径：没有记录就不会被算作"闭环已完成"——报告明确标 **PENDING**，评审一眼可见此项待补。

7. 运行与复现

7.1 一键部署（评审冷启动推荐）

```
cp .env.example .env          # 可选：填 GITHUB_TOKEN / LLM_API_KEY；不填也能跑
docker compose up --build
# 浏览器打开 http://127.0.0.1:5173
```

前端由 nginx 提供静态站点并把 `/api` 反向代理到后端容器（同源，免 CORS）。默认 `USE_CHROMA=false` 走确定性检索，**零模型下载、秒级启动**，适合离线评审。

7.2 本地开发

```
# 后端
cd backend && uv sync && uv run uvicorn backend.api.main:app --port 8000
# 前端
cd frontend && pnpm install && pnpm dev      # http://127.0.0.1:5173
```

7.3 接口

方法	路径	说明
POST	/api/profile	{url} → 仓库画像
POST	/api/search	{url, query, kind?} → 按视角路由的检索结果
POST	/api/guide	{url, use_llm} → Stage A–D 结构化指南
POST	/api/followup	三步追问：这步太粗 / 看不懂为什么 / 报错了
GET/POST	/api/llm-config	会话内 LLM 配置（不回显 Key）
GET	/health	存活检查

8. 项目路线图与开源计划

8.1 已交付

- 四段管线（M0 勘探 / M1 分层索引 / M2 结构化生成 / M3 分步引导）全部打通
- 四层评测体系（L1–L4）+ 5 个已人工复核的 golden 仓库
- Docker 一键部署、演示脚本、第三方许可披露、变更记录

8.2 后续计划

方向	内容
评测扩容	golden 集从 5 个扩到 15–20 个，覆盖更多语言与构建系统（Rust / Go / Java）
开源模型对比	用同一评估集对比开源模型与商业模型的输出质量（点题"AI + 开源"，不做过度承诺）
V2 架构视角升级	引入 tree-sitter 符号表，把"改哪个文件"从目录级细化到符号级
追问增强	报错定位接入更多语言生态的规则库
真实贡献	持续用本产品向小仓库提交 PR，滚动更新 L4 记录

8.3 开源计划

本作品以 MIT 许可开源。我们计划在赛事结束后公开仓库，接受外部 issue 与 PR—— 用自己做的工具来管理自己的新手贡献者，是这个项目最好的验证方式。

9. 隐私与授权

见 `privacy_and_license.md`。

附：文档索引

文档	内容
task_book.md	任务书（选题论证 / 技术方案 / 里程碑）
demo_script.md	现场演示脚本（3 个真实仓库 + 兜底方案）
video_script.md	演示视频分镜脚本
code_outline.md	代码大纲与接口约定
third_party_disclosure.md	第三方资源与许可披露
privacy_and_license.md	隐私与授权说明
cold_start_checklist.md	提交包冷启动演练记录
evaluation_report.md	评测报告（L2/L3 实测数字与归因）
changelog.md	变更记录（含修复根因与验证口径）
../benchmark/README.md	评测方法论与口径红线